SMART PRECISION AGRICULTURE

Full HLD, LLD, Data Flow & Project Blueprint

Engineering Blueprint v1.1
Based on the uploaded Smart Precision Agriculture plan

Core stack: React • FastAPI • MongoDB • ESP32 • MQTT • Python ML

A practical, farmer-first design for early risk detection and actionable alerts

1. Executive Summary

Smart Precision Agriculture is a farmer-focused decision-support application that combines field sensors, crop context, historical readings and machine-learning outputs to help farmers notice risks earlier and decide what to do next. It is more than a sensor dashboard: it follows a complete path from sensing and validation to prediction, safe decision-making, farmer alerts and feedback.

This blueprint is based on the uploaded Smart Precision Agriculture Engineering Blueprint. It preserves its stated core stack—ESP32, MQTT, FastAPI, MongoDB, Python ML and React—and expands it into an implementation-oriented plan. Items described as recommended are engineering choices to confirm during development, not requirements proven by the source.

- Primary users: farmers, agribusiness advisors, and the development/maintenance team.
- Core outcomes: monitor field conditions, estimate irrigation need, assess crop/environment risk, analyze leaf images where a trained model supports it, and deliver clear recommendations.
- MVP boundary: no drone spraying and no fully autonomous pump control without human override.
- Design principle: Sense → Validate → Store → Understand → Predict → Decide → Alert → Learn.

2. Goals, Scope & Assumptions

In scope

- Farmer account and localized responsive React/PWA interface.
- Farm and field registration, crop/variety, sowing date, growth stage, area and soil characteristics.
- ESP32/Arduino device registration, sensor health, calibration and last-seen status.
- Soil moisture, temperature, humidity and rainfall; optional pH and NPK sensors.
- Historical charts, offline buffering and later synchronization.
- Irrigation/water-need prediction, environmental/crop risk, and leaf-image disease classification when an appropriate model is available.
- Low/medium/high severity alerts, explanations, recommendations, feedback and optional push/SMS/voice channels.

Out of scope for MVP

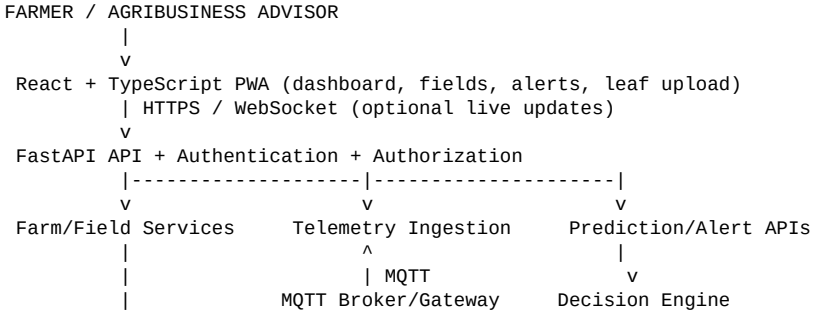
- Autonomous pesticide application or drone spraying.
- Pump actuation without human confirmation and safety interlocks.
- Claims of disease detection accuracy before field-representative evaluation.
- Guaranteed yield/cost analytics where source data is not available.

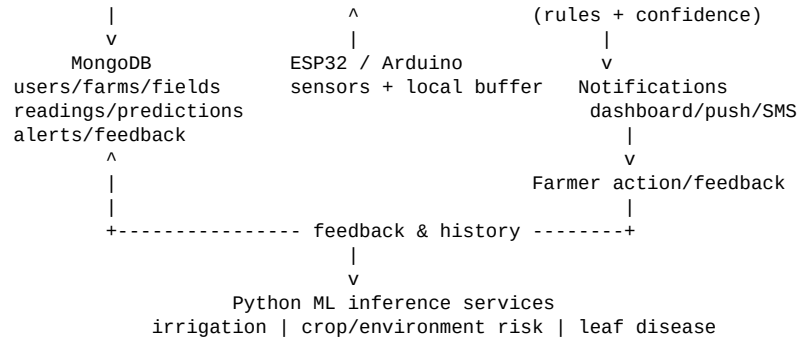
Working assumptions

- React frontend is deployed separately from the FastAPI backend.
- MongoDB stores operational metadata, sensor readings, predictions, alerts and feedback.
- MQTT is the primary telemetry transport; HTTP is a fallback for telemetry ingestion.
- Weather forecasts and SMS/push providers are optional integrations, not guaranteed dependencies.
- ML output is advisory. Confidence thresholds and agronomic rules gate what the farmer sees.

3. HLD — High-Level Design

The high-level design separates five concerns: field data collection, secure transport, backend/API orchestration, storage and ML inference, and farmer-facing decisions. Keeping these boundaries separate makes it easier to test hardware independently, replace a model, and add notification channels without rewriting the application.





Component responsibilities

Component	Responsibility
React / PWA	Farmer-friendly dashboard, farm/field forms, sensor charts, alerts, leaf upload, language and offline-friendly UX.
ESP32 / Arduino	Read sensors, calibrate/filter values, validate ranges, timestamp data, buffer offline readings and publish telemetry.
MQTT broker	Lightweight telemetry transport; authenticate devices and use TLS where supported.
FastAPI	Authentication, authorization, API validation, ingestion, orchestration, business rules and response formatting.
MongoDB	Persist users, farms, fields, devices, readings, predictions, alerts and farmer feedback.
Python ML	Inference for irrigation, crop/environment risk and leaf-image classification; record model version and confidence.
Decision engine	Combine prediction, confidence, crop stage, thresholds and safety rules; choose severity and recommendation.
Notification service	Deliver in-app alerts and optional push/SMS/voice messages; track delivery outcome.

Data acquisition, prediction and decision logic should remain separate. A model estimates a class, score or need; the decision engine decides whether and how to communicate it.

4. LLD — Low-Level Design

4.1 Telemetry processing sequence

```
Sensor read → calibration → range validation → noise filtering → timestamp → local buffer → MQTT publish → device authent
```

4.2 Device message contract

```
{
  "device_id": "ESP32-001",
  "field_id": "FIELD-001",
  "timestamp": "2026-10-07T07:30:00Z",
  "sequence": 1042,
  "soil_moisture": 22.4,
  "temperature": 31.5,
  "humidity": 78.2,
  "rainfall": 0,
  "ph": 6.7,
  "nitrogen": 42,
  "phosphorus": 28,
  "potassium": 35,
  "firmware_version": "1.0.0"
}
```

The values above are illustrative, not agronomic thresholds. Define sensor-specific units, calibration curves, valid ranges and missing-value behavior before field use. The sequence field is a recommended addition for duplicate detection and offline synchronization.

4.3 MQTT topics and delivery

- Telemetry topic: farm/{farm_id}/field/{field_id}/sensor/{device_id}.
- Use per-device credentials and broker ACLs so a device can publish only to its assigned topic.
- Use QoS 1 where delivery reliability is more important than avoiding duplicates; make the consumer idempotent.
- Retain unsent readings locally and remove them only after broker/backend acknowledgement.
- Use a device timestamp plus server-received timestamp; flag stale, future-dated or out-of-order samples.

4.4 Backend ingestion steps

1. Authenticate the device and verify its farm/field assignment.
2. Validate payload schema, numeric types, timestamps, units and allowed ranges.
3. Deduplicate using device_id plus sequence or a stable event identifier.
4. Persist the accepted raw reading and ingestion metadata.
5. Build rolling features from recent readings, crop metadata and optional weather context.
6. Run only the eligible model(s); capture model version, confidence, feature window and inference time.
7. Apply decision rules and safety constraints; do not promote low-confidence output to a high-impact action.
8. Persist prediction and alert with an explanation; deduplicate repeated active alerts.
9. Publish in-app notification and optionally enqueue push/SMS/voice delivery.
10. Record delivery status and accept farmer feedback for later evaluation.

4.5 Prediction/alert decision logic

```
if input_is_stale or required_features_missing:
    return status("insufficient_data", explain_missing_inputs=True)

prediction = model.predict(features)

if prediction.confidence < configured_min_confidence:
    save_prediction(status="needs_review", model_version=prediction.version)
    do_not_issue_high_impact_alert()

severity = rules.evaluate(
    prediction=prediction,
    crop=field.crop,
    growth_stage=field.growth_stage,
    recent_trends=features.trends
)

recommendation = recommendation_catalog.lookup(
    prediction_type=prediction.type,
    severity=severity,
    crop=field.crop,
    growth_stage=field.growth_stage
)

save_prediction_and_alert_atomically()
notify_if_new_or_escalated_alert()
```

Treat this as pseudocode. Thresholds, confidence cut-offs and recommendations must be configured and validated with agronomic expertise; they must not be copied blindly from a sample.

4.6 Error and retry behavior

- Malformed payload: reject, log a request/event ID and do not run inference.
- Database or network failure: retry with bounded exponential backoff; avoid duplicate writes.
- MQTT reconnect: publish buffered messages with original timestamps and stable event IDs.
- ML service unavailable: keep the reading, mark prediction pending/failed and retry asynchronously.
- Notification provider failure: retain the alert and retry delivery separately from prediction processing.
- No recent readings: show last-seen time and a clear stale/offline state instead of presenting old data as live.

5. Data Flow Diagrams

5.1 Context / Level 0

```
[ESP32 sensors] --telemetry--> [Smart Agriculture Platform] --alerts/recommendations--> [Farmer]
[Farmer] --farm setup, image, feedback--> [Smart Agriculture Platform]
[Weather provider, optional] --forecast--> [Platform]
[Platform] --optional SMS/push/voice--> [Farmer]
```

5.2 Level 1 — Operational flow

1. Capture: sensors read field conditions; firmware calibrates, filters and buffers.
2. Transport: MQTT carries telemetry; HTTP is the fallback path.
3. Validate: backend authenticates device, validates schema and deduplicates events.
4. Store: accepted readings and received timestamps are written to MongoDB.
5. Enrich: feature builder joins readings with crop, growth stage, history and optional weather.
6. Predict: ML services produce irrigation, crop-risk or disease outputs with confidence/version.
7. Decide: rule engine checks confidence, severity, context and safety conditions.
8. Communicate: alert and recommendation are saved; notification service delivers them.
9. Learn: farmer feedback and outcomes are stored for evaluation and future model improvement.

5.3 Leaf-image flow

```
Farmer captures/uploads leaf image → API authenticates user → file type/size checks → image quality check → preprocessing -
```

5.4 Offline synchronization

```
Network unavailable → ESP32 stores timestamped events locally → retry with backoff → network returns → publish buffered eve
```

5.5 Trust boundaries

- Device to broker: device identity, TLS, topic ACLs and replay/duplicate handling.
- Browser to API: HTTPS, user session/token, authorization, rate limiting and request validation.
- API to database: least-privilege database credentials and network restrictions.
- ML to decision engine: typed output schema, model version, confidence and bounded timeouts.
- Notification provider: store secrets securely, minimize farmer data and record delivery status.

6. Project Directory Structure

Suggested monorepo layout. It follows the uploaded blueprint and keeps the web application, API, ML, firmware, infrastructure and documentation in separate areas.

```
smart-agriculture/
■■■ frontend/
■   ■■■ public/
■   ■■■ src/
■   ■   ■■■ app/                      # routes and app shell
■   ■   ■■■ components/
■   ■   ■   ■■■ dashboard/
■   ■   ■   ■■■ charts/
■   ■   ■   ■■■ sensors/
■   ■   ■   ■■■ alerts/
■   ■   ■   ■■■ common/
■   ■   ■■■ pages/
■   ■   ■   ■■■ Dashboard.tsx
■   ■   ■   ■■■ Farms.tsx
■   ■   ■   ■■■ Fields.tsx
■   ■   ■   ■■■ Sensors.tsx
■   ■   ■   ■■■ DiseaseDetection.tsx
■   ■   ■   ■■■ Alerts.tsx
■   ■   ■   ■■■ Analytics.tsx
■   ■   ■■■ services/                 # API client and endpoint modules
■   ■   ■■■ hooks/
■   ■   ■■■ store/
■   ■   ■■■ types/
■   ■   ■■■ utils/
■   ■   ■■■ main.tsx
■   ■■■ package.json
■   ■■■ vite.config.ts
■■■ backend/
■   ■■■ app/
■   ■   ■■■ main.py
■   ■   ■■■ api/v1/
■   ■   ■   ■■■ auth.py
■   ■   ■   ■■■ farms.py
■   ■   ■   ■■■ fields.py
■   ■   ■   ■■■ sensors.py
■   ■   ■   ■■■ readings.py
■   ■   ■   ■■■ predictions.py
■   ■   ■   ■■■ diseases.py
■   ■   ■   ■■■ alerts.py
■   ■   ■   ■■■ recommendations.py
■   ■   ■   ■■■ analytics.py
■   ■   ■■■ core/                     # config, security, database, logging
■   ■   ■■■ models/                   # MongoDB document models
■   ■   ■■■ schemas/                  # request/response validation
■   ■   ■■■ services/                 # business workflows
```

```
■   ■   ■■■ repositories/             # database access
■   ■   ■■■ iot/                      # MQTT client, parser, device manager
■   ■   ■■■ decision/                 # rules, risk and action engine
■   ■   ■■■ notifications/             # in-app, push, SMS, voice adapters
■   ■   ■■■ workers/                  # background jobs/retries
■   ■■■ tests/
■   ■■■ requirements.txt
■   ■■■ Dockerfile
■■■ ml/
■   ■■■ common/                       # feature schemas and evaluation helpers
■   ■■■ disease_detection/            # train, inference, preprocessing
■   ■■■ irrigation_prediction/        # train, inference, features
■   ■■■ crop_risk/                    # train, inference, features
■   ■■■ datasets/                     # metadata only; large data ignored by Git
■   ■■■ models/                       # versioned artifacts or model registry refs
■   ■■■ evaluation/
■■■ firmware/
■   ■■■ esp32/
■   ■   ■■■ src/main.cpp
■   ■   ■■■ sensors/
■   ■   ■■■ mqtt/
■   ■   ■■■ storage/
■   ■   ■■■ config/
■   ■■■ arduino/
■■■ infrastructure/
■   ■■■ docker/
■   ■■■ nginx/
■   ■■■ mqtt/
■   ■■■ monitoring/
■■■ docs/
■   ■■■ HLD.md
■   ■■■ LLD.md
■   ■■■ DATA_FLOW.md
■   ■■■ API.md
■   ■■■ IOT_FIRMWARE.md
■   ■■■ ML_GOVERNANCE.md
■   ■■■ DATABASE.md
■■■ tests/
■   ■■■ integration/
■   ■■■ end_to_end/
■■■ docker-compose.yml
■■■ .env.example
■■■ .gitignore
■■■ README.md
```

Keep .env files, real credentials, private datasets and large model weights out of source control. Commit .env.example with variable names and safe placeholders only.

7. Database Design — MongoDB

Collection	Core fields and relationships
users	_id, name, phone/email, password_hash or identity-provider ID, language, role, created_at, updated_at.
farms	_id, owner_user_id, name, location, soil_type, area, created_at.
fields	_id, farm_id, name, crop, variety, sowing_date, growth_stage, area, soil_characteristics.
sensors	_id, field_id, device_id, sensor_types, units, status, calibration, firmware_version, last_seen, credential_ref.
sensor_readings	_id, event_id, device_id, sensor_id, field_id, event_timestamp, received_at, moisture, temperature, humidity, rainfall, pH, N, P, K, validation_status.
predictions	_id, field_id, prediction_type, created_at, risk_score, confidence, severity, status, explanation, feature_window, model_version.
alerts	_id, field_id, prediction_id, type, severity, message, recommendation, status, created_at, acknowledged_at, resolved_at, delivery_status.
feedback	_id, prediction_id, user_id, farmer_label, comment, created_at.
notification_jobs	_id, alert_id, channel, recipient_ref, status, attempts, next_attempt_at, delivered_at, provider_message_id.

Relationships

- User 1→many Farms; Farm 1→many Fields; Field 1→many Sensors and Readings.
- Field 1→many Predictions and Alerts; Prediction 1→many Feedback entries.
- Alert 1→many notification jobs for delivery attempts and channels.

Recommended indexes and retention

- sensor_readings: compound index on { field_id: 1, event_timestamp: -1 }; unique index on event_id or { device_id: 1, sequence: 1 }.
- alerts: { field_id: 1, created_at: -1 } and { status: 1, severity: 1 }.
- sensors: unique { device_id: 1 }; index on { status: 1, last_seen: -1 }.
- predictions: { field_id: 1, created_at: -1 } and { model_version: 1, prediction_type: 1 }.
- Set a documented retention/archival policy for high-volume telemetry; choose TTL only if deleting older readings is acceptable. Back up before enabling automatic expiry.

8. REST API Blueprint

Method	Endpoint	Purpose
POST	/api/v1/auth/login	Authenticate farmer and issue session/token.
GET / POST	/api/v1/farms	List farms / create a farm.
GET / POST	/api/v1/farms/{farm_id}/fields	List or create fields within a farm.
GET / PATCH	/api/v1/fields/{field_id}	Read/update field details.
POST	/api/v1/sensors/register	Register or pair a device.
GET /	/api/v1/sensors	List assigned sensors and health status.
POST	/api/v1/readings	HTTP telemetry fallback; validate and deduplicate.
GET	/api/v1/readings?field_id=...&from=...&to=...	Read history with pagination and time range.
GET	/api/v1/predictions/{field_id}	Latest predictions for a field.
POST multipart	/api/v1/disease/analyze	Upload a leaf image and request inference.
GET	/api/v1/alerts	List authorized user's alerts.
PATCH	/api/v1/alerts/{alert_id}	Acknowledge or resolve an alert.
POST	/api/v1/feedback	Submit feedback for a prediction.
GET	/api/v1/analytics/{field_id}	Field trends and supported analytics.
GET	/health	Liveness/health endpoint; avoid leaking secrets.

API conventions

- Use a versioned prefix (/api/v1), HTTPS, authentication and per-resource authorization.
- Validate all request bodies and uploaded images; set request-size and rate limits.
- Use pagination for readings and alerts; cap time ranges and return server timestamps.
- Return consistent errors with code, message, request_id and optional field_errors.
- Recommended statuses: 200 read/update, 201 create, 202 accepted asynchronous work, 400 invalid request, 401 unauthenticated, 403 unauthorized, 404 missing resource, 409 duplicate/conflict, 413 oversized upload, 422 schema validation, 429 rate limit, 500 unexpected error.
- Use idempotency keys for retried device submissions and asynchronous image-analysis requests where practical.

9. ML & Model Governance

Disease vision pipeline

```
Leaf image → file/quality checks → resize and normalize → trained CNN/transfer-learning model → disease class + confidence
```

Irrigation model features

- Current and rolling soil moisture; temperature and humidity; recent rainfall and forecast when available.
- Crop, growth stage, soil type and historical irrigation response.
- Missingness, sensor quality and reading age should be explicit features or inference eligibility checks.

Crop/environment risk features

- Temperature/humidity patterns, rainfall persistence and recent sensor trends.

- Crop and growth stage, historical disease observations and optional weather context.
- A single reading should not be treated as sufficient evidence for every risk type.

Governance requirements

- Store model_version, inference timestamp, confidence and input/feature window with every prediction.
- Keep training, validation and held-out test data separate; track precision, recall, confusion matrix and false-positive/false-negative rates.
- Evaluate across crops, seasons, devices and lighting/field conditions represented in intended use.
- Monitor drift and collect farmer feedback as a candidate label, not automatically as ground truth.
- Low-confidence or out-of-distribution inputs should produce an uncertain/needs-review result rather than strong advice.
- Recommendations and high-impact advice require agronomic review and field validation.

10. Alert & Recommendation Engine

Severity	Expected behavior
LOW	Show trend or information in the dashboard; avoid disruptive notifications.
MEDIUM	Show a dashboard warning and a practical inspection/action suggestion.
HIGH	Send immediate configured notification with a clear reason; require human judgment for high-impact actions.

- Each alert should explain what was observed, why it matters, confidence/limitations, when it was detected and what the farmer can do next.
- Deduplicate repeated alerts for the same field/type while an alert remains active; update or escalate rather than spamming.
- Separate a model prediction from a recommendation. The model estimates risk; decision rules and agronomic guidance determine what is communicated.
- Support acknowledgement, resolution, feedback and an audit trail of changes.
- Do not command pumps or other equipment automatically in the MVP; any future actuation needs explicit confirmation, interlocks, fail-safe defaults and field testing.

11. Farmer UX & Accessibility

Main screens

- Dashboard: farm health summary, current sensor cards, active alerts, water recommendation, disease risk and weather/context when available.
- Farm and Field Details: crop/growth stage, live and historical readings, prediction timeline, field/device status and last-seen time.
- Sensors: device pairing, calibration, signal/health status and offline indicator.
- Disease Detection: camera/upload, image-quality result, classification confidence, limitations and next step.
- Alerts: severity, explanation, acknowledgement/resolution and notification status.
- Analytics: water usage, risk trends, alert history and yield/cost only when supporting data exists.
- Settings: language, units, notification preferences, account and permissions.

Use plain language and put the recommended next action first; technical readings can be available on demand. Support regional languages and optional audio prompts. Clearly distinguish live, delayed, missing and predicted values.

12. IoT & Hardware

Device	Role
ESP32	Low-cost field controller with Wi-Fi/Bluetooth.
Capacitive soil moisture sensor	Estimate soil water availability after calibration for the soil/sensor combination.
Temperature/humidity sensor	Measure local microclimate.
Rain sensor or weather input	Provide rainfall context.
RS485 NPK probe (optional)	Nutrient measurement when supported and calibrated.
pH probe (optional)	Soil acidity measurement.
Flow sensor (optional)	Verify irrigation delivery if installed.
Mobile/leaf camera	Capture disease-image input.
IP65 enclosure	Help protect electronics from outdoor exposure.

For larger or remote fields, consider LoRa after measuring field distance, obstacles, power and gateway placement. Choose sensor models and power architecture only after field trials. Plan for corrosion, heat, rain, dust, calibration and periodic maintenance.

13. Security, Reliability & Operations

- Provision unique device credentials and restrict MQTT topics by device identity.
- Use TLS for internet-facing MQTT/HTTP where supported; keep secrets outside source control.
- Use role-based permissions for farmers, advisors and administrators; enforce ownership checks on every farm/field resource.
- Validate all sensor payloads on the server; never trust device-side checks alone.
- Rate-limit public APIs and image analysis; constrain image type, size and processing time.
- Use least-privilege MongoDB credentials, backups and tested restore procedures.
- Use bounded retries with exponential backoff and idempotent ingestion.
- Monitor device last-seen, sensor anomalies, API errors, queue depth, inference latency and notification failures.
- Keep structured logs with request/event IDs; audit alert changes and administrative actions.
- Expose health checks and operational metrics without leaking credentials or farmer data.

14. Testing Strategy

Layer	Key checks
Firmware	Sensor accuracy, calibration, retry, offline buffer and reconnect.
IoT/MQTT	Authentication, ACLs, malformed payloads, duplicate messages, QoS and reconnect behavior.
Backend	Authentication, authorization, schema validation, rate limits, persistence and error handling.
ML	Held-out accuracy, precision/recall, confidence thresholds, class imbalance and model versioning.
Vision	Blur, lighting, angle, unsupported crops and oversized/invalid images.
Decision engine	Boundary conditions, stale/missing signals, conflicting readings, alert deduplication and escalation.
Integration	ESP32 → broker → API/consumer → DB → ML → decision → alert → UI.
Field	Heat, rain, dust, network loss, long-duration stability and sensor drift.
Security	Cross-user data access, credential leakage, upload abuse, rate limits and audit trails.

Acceptance criterion: a farmer must be able to identify whether data is current, understand why an alert appeared, see a practical next step, and provide feedback. Hardware and ML accuracy must be measured in field conditions before production claims are made.

15. Deployment & Environments

Suggested environment split

- Development: local React app, FastAPI, MongoDB development database and local/test MQTT broker.
- Staging: production-like deployment, test devices, synthetic or consented data, and separate credentials.
- Production: HTTPS frontend/API, secured MQTT broker, managed or backed-up MongoDB, model artifacts/versioning, monitoring and alert delivery configuration.

Deployment checklist

- Containerize frontend/API/worker services where useful; document local startup commands in README.
- Store secrets in deployment environment/secret manager; never in frontend variables or Git.
- Configure CORS to known frontend origins and set trusted proxy/HTTPS settings correctly.
- Separate dev/staging/prod databases and MQTT credentials.
- Set health checks, logs, metrics, backups, restore drills and dependency update process.
- Deploy ML models as versioned artifacts; test rollback to a previous known-good model.
- Verify rural/offline behavior and notification fallback before launch.

16. Implementation Roadmap

Phase	Deliverables	Exit condition
1. Foundation	Repository, React app, FastAPI, MongoDB, config, authentication, CI/tests.	A farmer can sign in and create a farm/field.
2. IoT MVP	ESP32 sensor reads, MQTT, device pairing, telemetry validation and persistence.	Verified readings appear with timestamps and device health.
3. Dashboard	Current sensor cards, history charts, field management and offline/stale indicators.	Farmer can inspect a field and understand data freshness.
4. Rules & Alerts	Threshold/trend rules, severity, deduplication and in-app alerts.	Test cases create, update and resolve alerts predictably.
5. ML v1	Irrigation/environmental risk model, evaluation and model versioning.	Model passes held-out evaluation and uncertainty behavior is tested.
6. Disease Vision	Leaf upload, quality checks, inference and confidence-aware output.	Supported-image flow is tested; limitations are clear.
7. Feedback	Farmer feedback, notification channels and evaluation reporting.	Feedback is linked to predictions and delivery is observable.
8. Production	Security hardening, monitoring, backups, offline sync and field validation.	Operational checklist passes in a pilot environment.

The source document proposes a four-week MVP across architecture/component selection, hardware setup, ML/backend development, and integration/testing. Treat that as an initial prototype target; the complete production roadmap above is incremental and may require more time depending on hardware access, dataset readiness and field validation.

17. Risks & Mitigations

Risk	Mitigation
Weak rural connectivity	Local buffering, retry with backoff, stale-data indicators and synchronization tests.
Outdoor hardware exposure	IP-rated enclosure, corrosion-resistant components, maintenance and calibration schedule.
ML misclassification	Confidence gates, held-out evaluation, human review and farmer feedback.
False/repeated alerts	Trend-based rules, deduplication, severity policy and audit history.
Bad sensor readings	Calibration, range validation, anomaly detection and explicit sensor-health state.
Sensitive credentials/data	Unique device credentials, HTTPS/TLS, least privilege, secret management and access control.
Premature automation	No autonomous high-impact control in MVP; human override and safety interlocks for any future actuator.

18. Build-Ready Checklist

- Hardware sensors are calibrated and validated under field conditions.
- Data survives network loss and synchronizes without uncontrolled duplication.
- Device and user authentication, ownership checks and TLS are configured.
- Telemetry, farm, prediction and alert APIs have automated tests.
- MongoDB indexes and telemetry retention policy are documented.
- ML models have held-out evaluation, confidence thresholds and version tracking.
- Alerts are actionable, deduplicated and severity-based.
- The UI explains current vs stale data in plain language and supports the target languages.
- Logs, metrics, backups and restore checks exist.
- Agronomic recommendations are reviewed before high-impact advice is released.

Definition of done

The MVP is ready for a controlled pilot when a real sensor event can be traced from device to database to dashboard, a test risk can create a justified alert, duplicate/offline events are handled safely, and the farmer can acknowledge the alert and submit feedback. ML disease and irrigation claims remain provisional until evaluated with suitable data.

19. API Payload Examples

Create farm request

```
{
  "name": "North Field Farm",
  "location": {"district": "Example District", "state": "Uttar Pradesh"},
  "soil_type": "loamy",
  "area_hectares": 1.2
}
```

Alert response

```
{
  "id": "alert_123",
  "field_id": "field_123",
  "type": "irrigation_need",
  "severity": "medium",
  "status": "active",
  "message": "Soil moisture has remained below the configured field threshold.",
  "recommendation": "Inspect soil moisture and recent rainfall before deciding whether to irrigate.",
  "confidence": 0.82,
  "model_version": "irrigation-v1",
  "created_at": "2026-10-25T10:00:00Z"
}
```

These are illustrative contracts only. Production schemas must define units, validation bounds, localization behavior, nullability and authorization rules.

20. Final Architecture Principle

Build the system as a reliable field-specific decision-support platform, not merely an IoT dashboard. Preserve the separation between sensor acquisition, data validation, persistence, ML prediction, decision rules, notification delivery and farmer feedback. Start with trusted telemetry and transparent rule-based alerts; add ML only when the collected data and evaluation process support it.

Final sequence: Sense → Validate → Store → Understand → Predict → Decide → Alert → Learn.